

Explainable Deep Reinforcement Learning for Portfolio Management: An Empirical Approach

Mao Guan

Computer Science, Columbia University
New York City, NY
mg3844@columbia.edu

Xiao-Yang Liu

Electrical Engineering, Columbia University
New York City, NY
xl2427@columbia.edu

Abstract—Deep reinforcement learning (DRL) has been widely studied in the portfolio management task. However, it is challenging to understand a DRL-based trading strategy because of the black-box nature of deep neural networks. In this paper, we propose an empirical approach to explain the strategies of DRL agents for the portfolio management task. First, we use a *linear model in hindsight* as the *reference model*, which finds the best portfolio weights by assuming knowing actual stock returns *in foresight*. In particular, we use the coefficients of a linear model in hindsight as the *reference feature weights*. Secondly, for DRL agents, we use integrated gradients to define the feature weights, which are the coefficients between reward and features under a linear regression model. Thirdly, we study the prediction power in two cases, single-step prediction and multi-step prediction. In particular, we quantify the prediction power by calculating the linear correlations between the feature weights of a DRL agent and the *reference feature weights*, and similarly for machine learning methods. Finally, we evaluate a portfolio management task on Dow Jones 30 constituent stocks during 01/01/2009 to 09/01/2021. Our approach empirically reveals that a DRL agent exhibits a stronger multi-step prediction power than machine learning methods.

Index Terms—Explainable deep reinforcement learning, Integrated Gradient, linear model in hindsight, portfolio management

I. INTRODUCTION

The explanation [Jae+20] of a portfolio management strategy is important to investment banks, asset management companies and hedge funds. It helps traders understand the potential risk of a certain strategy. However, it is challenging to explain a DRL-based portfolio management strategy due to the black-box nature of deep neural networks.

Existing DRL-based portfolio management works focus on enhancing the performance. A typical DRL approach of portfolio management consists of three steps as described in [Liu+20; Liu+21c; Liu+21a; Liu+21b; Li+21]. First, select a pool of possibly risky assets. Secondly, specify the state space, action space and reward function of the DRL agent. Finally, train a DRL agent to learn a portfolio management strategy. Such a practical approach, however, does not provide explanation to the portfolio management strategy.

In recent years, explainable deep reinforcement learning methods have been widely studied. Quantifying how much a change in input would influence the output is important to understand what contributes to the decision-making processes of the DRL agents. Thus, saliency maps [TG20] are adopted

to provide explanation. However, these approaches are mainly available in computer vision, natural language processing and games [ACJ19; HCD21; Mad+20]. They have not been widely applied in financial applications yet. Some researchers [Con+21] explain the DRL based portfolio management strategy using an attention model. However, it does not explain the decision-making process of a DRL agent in a proper financial context.

In this paper, we take an empirical approach to explain the portfolio management strategy of DRL agents. Our contributions are summarized as follows

- We propose a novel empirical approach to understand the strategies of DRL agents for the portfolio management task. In particular, we use the coefficients of a *linear model in hindsight* as the *reference feature weights*.
- For a deep reinforcement learning strategy, we use integrated gradients to define the feature weights, which are the coefficients between the reward and features under a linear regression model.
- We quantify the prediction power by calculating the linear correlations between the feature weights of a DRL agent and the reference feature weights, and similarly for conventional machine learning methods. Moreover, we consider both the single-step case and multiple-step case.
- We evaluate our approach on a portfolio management task with Dow Jones 30 constituent stocks during 01/01/2009 to 09/01/2021. Our approach empirically explains that a DRL agent achieves better trading performance because of its stronger multi-step prediction power.

The remainder of this paper is organized as follows. In Section 2, we review existing works on the explainable deep reinforcement learning. In Section 3, we describe the problem formulation of a DRL-based portfolio management task. In Section 4, we present the proposed explanation method. In Section 5, we show quantitative experimental results of our empirical approach. Finally, the conclusion and future work are given in Section 6.

II. RELATED WORKS

Gradient based explanation methods are widely adopted in the saliency maps [TG20], which quantify how much a change in input would influence the output. We review the related

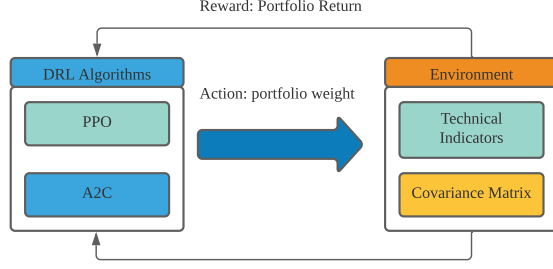


Fig. 1. Overview of a portfolio management task that uses deep reinforcement learning.

works of gradient based explanation for deep reinforcement learning. leftmargin=*

- **Gradient $\odot$ Input** [SGK17] is the element-wise product of the gradient and the input. It provides explanation by visualizing the product as heatmap.
- **Integrated Gradient (IG)** [STY17]. It integrates the gradient of the output with respect to input features. For an input $x \in \mathbb{R}^n$, the i -th entry of integrated gradient is defined as

$$\text{IG}(x)_i \triangleq (x_i - x'_i) \times \int_{z=0}^1 \frac{\partial F(x' + z \cdot (x - x'))}{\partial x_i} dz, \quad (1)$$

where $F(\cdot)$ denotes a DRL model, x' is a perturbed version of x , say replacing all entries with zeros. It explains the relationship between a model's predictions in terms of its features.

- **Guided Backpropagation (GBP)** computes the gradient of the target output with respect to the input [Spr+15; ZF14], and it treats negative gradients as zeros. It provides explanation by visualizing the gradients.
- **Guided GradCAM** [Sel+16]. It uses the class-specific gradient and the final layer of a convolutional neural network to produce a coarse localization map of the important regions in an image. It provides explanation using a gradient-weighted map.
- **SmoothGrad (SG)** [Smi+17; STY17]. It creates noisy copies of an input image then averages gradients with respect to these copies. It provides explanation using visual map to identify pixels that strongly influence the final result.

Although these gradient based explanation methods are popular, they have not been directly applicable to the portfolio management task yet. Other researchers [Con+21] explain the DRL based portfolio management using an attention model. However, it does not explain the decision-making process of DRL agent in a proper financial context.

III. PORTFOLIO MANAGEMENT USING DEEP REINFORCEMENT LEARNING

We first describe a portfolio management task using a DRL agent. Then we define the feature weights using integrated gradients.

A. Portfolio Management Task

Consider a portfolio with N risky assets over T time slots, the portfolio management task aims to maximize profit and minimize risk. Let $p(t) \in \mathbb{R}^N$ denotes the closing prices of all assets at time slot $t = 1, \dots, T$.¹ The *price relative vector* $y(t) \in \mathbb{R}^N$ is defined as the element-wise division of $p(t)$ by $p(t-1)$:

$$y(t) \triangleq \left[\frac{p_1(t)}{p_1(t-1)}, \frac{p_2(t)}{p_2(t-1)}, \dots, \frac{p_N(t)}{p_N(t-1)} \right]^\top, \quad t = 1, \dots, T, \quad (2)$$

where $p(0) \in \mathbb{R}^N$ is the vector of opening prices at $t = 1$.

Let $w(t) \in \mathbb{R}^N$ denotes the portfolio weights, which is updated at the beginning of time slot t . Let $v(t) \in \mathbb{R}$ denotes the portfolio value at the beginning of time slot $t+1$.² Ignoring the transaction cost, we have the *relative portfolio value* as the ratio between the portfolio value at the ending of time slot t and that at the beginning of time slot t ,

$$\frac{v(t)}{v(t-1)} = w(t)^\top y(t), \quad (3)$$

where $v(0)$ is the initial capital. The *rate of portfolio return* is

$$\rho(t) \triangleq \frac{v(t)}{v(t-1)} - 1 = w(t)^\top y(t) - 1, \quad (4)$$

while correspondingly the *logarithmic rate of portfolio return* is

$$r(t) \triangleq \ln \frac{v(t)}{v(t-1)} = \ln(w(t)^\top y(t)). \quad (5)$$

The risk of a portfolio is defined as the variance of the rate of portfolio return $\rho(t)$:

$$\begin{aligned} \text{Risk}(t) &\triangleq \text{Var}(\rho(t)) = \text{Var}(w(t)^\top y(t) - 1) \\ &= \text{Var}(w(t)^\top y(t)) = w(t)^\top \text{Cov}(y(t)) w(t) \\ &= w(t)^\top \Sigma(t) w(t), \end{aligned} \quad (6)$$

where $\Sigma(t) = \text{Cov}(y(t)) \in \mathbb{R}^{N \times N}$ is the covariance matrix of the stock returns at the end of time slot t . If there is no transaction cost, the final portfolio value is

$$v(T) = v(0) \exp \left(\sum_{t=1}^T r(t) \right) = v(0) \prod_{t=1}^T w(t)^\top y(t). \quad (7)$$

The portfolio management task [Boy+17; Wik21c] aims to find a portfolio weight vector $w^*(t) \in \mathbb{R}^N$ such that

$$\begin{aligned} w^*(t) &\triangleq \underset{w(t)}{\text{argmax}} \quad w^\top(t) y(t) - \lambda w^\top(t) \Sigma(t) w(t), \\ \text{s.t.} \quad &\sum_{i=1}^N w_i(t) = 1, \quad w_i(t) \in [0, 1], \quad t = 1, \dots, T, \end{aligned} \quad (8)$$

where $\lambda > 0$ is the risk aversion parameter. Since $y(t)$ and $\Sigma(t)$ are revealed at the end of time slot t . We estimate them at the the beginning of time slot t .

¹For continuous markets, the closing prices at time slot t is also the opening prices for time slot $t+1$.

²Similarly $v(t)$ is also the portfolio value at the ending of time slot t .

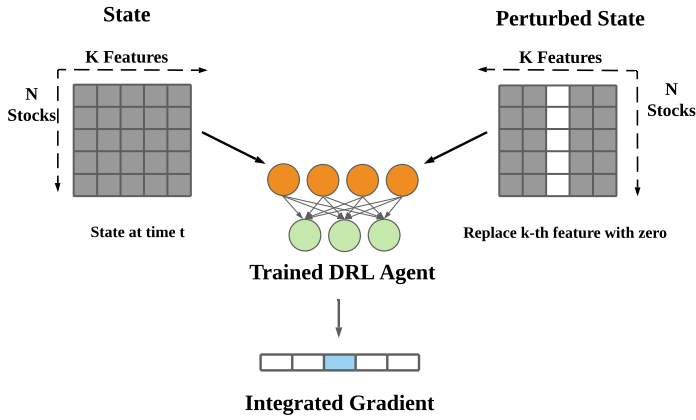


Fig. 2. Feature weights of a trained DRL agent.

We use $\hat{y}(t) \in \mathbb{R}^N$ to estimate the price relative vector $y(t)$ in (8) by applying a regression model on predictive financial features [FGX17] based on Capital Asset Pricing Model (CAPM) [FF04]. We use $\hat{\Sigma}(t)$, the sample covariance matrix, to estimate covariance matrix $\Sigma(t)$ in (8) using historical data.

Then, at the beginning of time slot t , our goal is to find optimal portfolio weights

$$w^*(t) \triangleq \underset{w(t)}{\operatorname{argmax}} \quad w^\top(t) \hat{y}(t) - \lambda w^\top(t) \hat{\Sigma}(t) w(t),$$

$$\text{s.t.} \quad \sum_{i=1}^N w_i(t) = 1, \quad w_i(t) \in [0, 1], \quad t = 1, \dots, T. \quad (9)$$

B. Deep Reinforcement Learning for Portfolio Management

We describe how to use deep reinforcement learning algorithms for the portfolio management task, by specifying the state space, action space and reward function. We use a similar setting as in the open-source FinRL library [Liu+20][Liu+21c].

State space $\mathcal{S}$ describes an agent's perception of a market. The state at the beginning of time slot t is

$$s(t) = [f^1(t), \dots, f^K(t), \hat{\Sigma}(t)] \in \mathbb{R}^{N \times (N+K)}, \quad t = 1, \dots, T, \quad (10)$$

where $f^k(t) \in \mathbb{R}^N$ denotes the vector for the k -th feature at the beginning of time slot t .

Action space $\mathcal{A}$ describes the allowed actions an agent can take at a state. In our task, the action $w(t) \in \mathbb{R}^N$ corresponds to the portfolio weight vector decided at the beginning of time slot t and should satisfy the constraints in (9). We use a softmax layer as the last layer to meet the constraints.

Reward function. The reward function $r(s(t), w(t), s(t+1))$ is the incentive for an agent to learn a profitable policy.

We use the logarithmic rate of portfolio return in (5) as the reward,

$$r(s(t), w(t), s(t+1)) = \ln(w^\top(t) \cdot y(t)). \quad (11)$$

The agent takes $s(t)$ as input at the beginning of time slot t and output $w(t)$ as the portfolio weight vector.

DRL algorithms. We use two popular deep reinforcement learning algorithms: Advantage Actor Critic (A2C) [Mni+16] and Proximal Policy Optimization (PPO) [Sch+17]. A2C [Mni+16] utilizes an advantage function to reduce the variance of the policy gradient. Its objective function is

$$\nabla J_\theta(\theta) = \mathbb{E} \left[\sum_{t=1}^T \nabla_\theta \log \pi_\theta(w(t)|s(t)) A(s(t), w(t)) \right], \quad (12)$$

where $\pi_\theta(w(t)|s(t))$ is the policy network parameterized by θ and $A(s(t), w(t))$ is an advantage function defined as follows

$$A(s(t), w(t)) = Q(s(t), w(t)) - V(s(t)) \\ = r(s(t), w(t), s(t+1)) + \gamma V(s(t+1)) - V(s(t)), \quad (13)$$

where $Q(s(t), w(t))$ is the expected reward at state $s(t)$ when taking action $w(t)$, $V(s(t))$ is the value function, $\gamma \in (0, 1]$ is a discount factor. PPO [Sch+17] is used to control the policy gradient update and to ensure that the new policy will be close to the previous one. It uses a surrogate objective function

$$J^{\text{CLIP}}(\theta) = \mathbb{E}(t) [\min(R_t(\theta) \hat{A}(s(t), w(t)), \text{clip}(R_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}(s(t), w(t))), \quad (14)$$

where $R_t(\theta) \triangleq \frac{\pi_\theta(w(t)|s(t))}{\pi_{\theta_{old}}(w(t)|s(t))}$ is the probability ratio between new and old policies, $\hat{A}(s(t), w(t))$ is the estimated advantage function, and the clip function $\text{clip}(R_t(\theta), 1-\epsilon, 1+\epsilon)$ truncates the ratio $R_t(\theta)$ to be within the range $[1-\epsilon, 1+\epsilon]$.

C. Feature Weights Using Integrated Gradients

We use the integrated gradients in (1) to measure the feature weights [STY17; Tom+20]. For a trained DRL agent, the integrated gradient [STY17] under policy π for the k -th feature

of the i -th asset is defined as

$$\begin{aligned}
& IG(f^k(t))_i \\
&= (f^k(t)_i - f^{k'}(t)_i) \\
&\quad \times \int_{z=0}^1 \frac{\partial Q^\pi(s'_k(t) + z \cdot (s(t) - s'_k(t)), w(t))}{\partial f^k(t)_i} dz \\
&= f^k(t)_i \cdot \frac{\partial Q^\pi(s'_k(t) + z^{k,i} \cdot (s(t) - s'_k(t)), w(t))}{\partial f^k(t)_i} \cdot (1 - 0) \\
&= f^k(t)_i \\
&\quad \cdot \frac{\partial \mathbb{E} \left[\sum_{l=0}^{\infty} \gamma^l \cdot r(s^{k,i}(t+l), w(t+l), s^{k,i}(t+l+1)) | s^{k,i}(t), w(t) \right]}{\partial f^k(t)_i} \\
&= f^k(t)_i \cdot \sum_{l=0}^{\infty} \gamma^l \cdot \frac{\partial \mathbb{E} [\ln(w^\top(t+l) \cdot y(t+l)) | s^{k,i}(t), w(t)]}{\partial f^k(t)_i} \\
&\approx f^k(t)_i \cdot \sum_{l=0}^{\infty} \gamma^l \cdot \frac{\partial \mathbb{E} [w^\top(t+l) \cdot y(t+l) - 1 | s^{k,i}(t), w(t)]}{\partial f^k(t)_i} \\
&= f^k(t)_i \cdot \sum_{l=0}^{\infty} \gamma^l \cdot \frac{\partial \mathbb{E} [w^\top(t+l) \cdot y(t+l) | s^{k,i}(t), w(t)]}{\partial f^k(t)_i}, \tag{15}
\end{aligned}$$

where the first equality holds by definition in (1), the second equality holds because of the mean value theorem [Wik21b], the third equality holds because

$$\begin{aligned}
& Q^\pi(s(t), w(t)) \triangleq \\
& \mathbb{E} \left[\sum_{l=0}^{\infty} \gamma^l \cdot r(s(t+l), w(t+l), s(t+l+1)) | s(t), w(t) \right], \tag{16}
\end{aligned}$$

the approximation holds because $\ln(w^\top(t) \cdot y(t)) \approx w^\top(t) \cdot y(t) - 1$ when $w^\top(t) \cdot y(t)$ is close to 1. $s'_k(t) \in \mathbb{R}^{N \times (N+K)}$ is a perturbed version of $s(t)$ by replacing the k -th feature with an all-zero vector. $s^{k,i}(t)$ is a linear combination of original state and perturbed state $s^{k,i}(t) \triangleq s'_k(t) + z^{k,i} \cdot (s(t) - s'_k(t))$, where $z^{k,i} \in [0, 1]$.

IV. EXPLANATION METHOD

We propose an empirical approach to explain the portfolio management task that uses a trained DRL agent.

A. Overview of Our Empirical Approach

Our empirical approach consists of three parts.

- First, we study the portfolio management strategy using feature weights, which quantify the relationship between the reward (say, portfolio return) and the input (say, features). In particular, we use the coefficients of a *linear model in hindsight* as the reference feature weights.
- Then, for the deep reinforcement learning strategy, we use integrated gradients to define the feature weights, which are the coefficients between reward and features under a linear regression model
- Finally, we quantify the prediction power by calculating the linear correlations between the coefficients of a DRL agent and the reference feature weights, and similarly for conventional machine learning methods. Moreover, we consider both the single-step case and multiple-step case.

B. Reference Feature Weights

For the portfolio management task, we use a *linear model in hindsight* as a reference model. For a *linear model in hindsight*, a demon would optimize the portfolio [BSN20] with actual stock returns and the *actual sample covariance matrix*. It is the upper bound performance that any linear predictive model would have been able to achieve.

The *portfolio value relative vector* is the element-wise product of weight and price relative vectors, $q(t) \triangleq w(t) \odot y(t) \in \mathbb{R}^N$, where $w(t)$ is the optimal portfolio weight. We represent it as a linear regression model as follows

$$q(t) = \beta_0(t) \cdot [1, \dots, 1]^\top + \beta_1(t) \cdot f^1(t) + \dots + \beta_K(t) \cdot f^K(t) + \epsilon(t), \tag{17}$$

where $\beta_k(t) \in \mathbb{R}$ is regression coefficient of the k -th feature. $\epsilon(t) \in \mathbb{R}^N$ is the error vector, where the elements are assumed to be independent and normally distributed.

We define the *reference feature weights* as

$$\beta(t) \triangleq [\beta(t)_1, \beta(t)_2, \dots, \beta(t)_K]^\top \in \mathbb{R}^K, \text{ where}$$

$$\beta(t)_k = \sum_{i=1}^N \beta_k(t) \cdot f^k(t)_i, \tag{18}$$

is the inner product of $\frac{\partial(q^\top(t) \cdot 1)}{\partial f^k(t)} = \beta_k(t) \cdot [1, \dots, 1]^\top$ and $f^k(t)$ that characterizes the total contribution of the k -th feature to the portfolio value at time t .

C. Feature Weights for DRL Trading Agent

For a DRL agent in portfolio management task, at the beginning of a trading slot t , it takes the feature vectors and co-variance matrix as input. Then it outputs an action vector, which is the portfolio weight vector $w(t)$. We also represent it as a linear regression model,

$$q(t) = c_0(t) \cdot [1, \dots, 1]^\top + c_1(t) \cdot f^1(t) + \dots + c_K(t) \cdot f^K(t) + \epsilon(t). \tag{19}$$

As Fig. 2 shows, for the decision-making process of a DRL agent, we define the feature weights for the k -th feature as

$$\begin{aligned}
M^\pi(t)_k &\triangleq \sum_{i=1}^N IG(f^k(t))_i \\
&\approx \sum_{i=1}^N f^k(t)_i \cdot \sum_{l=0}^{\infty} \gamma^l \cdot \frac{\partial \mathbb{E} [w^\top(t+l) \cdot y(t+l) | s^{k,i}(t), w(t)]}{\partial f^k(t)_i} \\
&= \sum_{i=1}^N f^k(t)_i \cdot \sum_{l=0}^{\infty} \gamma^l \cdot \mathbb{E} \left[c_k(t+l) \frac{\partial f^k(t+l)_i}{\partial f^k(t)_i} | s^{k,i}(t), w(t) \right], \tag{20}
\end{aligned}$$

where the last equality holds due to the fact that $\frac{\partial w^\top(t+l) \cdot y(t+l)}{\partial f^k(t)_i}$ is continuous and $w^\top(t+l) \cdot y(t+l)$ is bounded for any t [Wik21a; Wik21b].

Assuming the time dependency of features on stocks follows the power law, i.e., $\frac{\partial f^k(t+l)_i}{\partial f^k(t)_i} = l^{-\alpha}$, where $\alpha \in \mathbb{R}^+$, for $l \geq 1$, then the feature weights are

$$\begin{aligned}
M^\pi(t)_k &= \sum_{i=1}^N f^k(t)_i \cdot \sum_{l=0}^{\infty} \gamma^l \cdot \mathbb{E} \left[c_k(t+l) \frac{\partial f^k(t+l)_i}{\partial f^k(t)_i} | s^{k,i}(t), w(t) \right] \\
&= \sum_{i=1}^N f^k(t)_i \cdot \mathbb{E} \left[c_k(t) + \sum_{l=1}^{\infty} \gamma^l \cdot l^{-\alpha} \cdot c_k(t+l) | s^{k,i}(t), w(t) \right] \\
&= \sum_{i=1}^N f^k(t)_i \cdot \mathbb{E} \left[c_k(t) + \sum_{l=1}^{\infty} \gamma^l \cdot l^{-\alpha} \cdot c_k(t+l) | s^{k,i}(t), w(t) \right] \quad (21)
\end{aligned}$$

Notice that $M^\pi(t)_k$ has a similar form as $\beta(t)_k$ in (18). The $\beta_k(t)$ are replaced by $\mathbb{E} [c_k(t) + \sum_{l=1}^{\infty} \gamma^l \cdot l^{-\alpha} \cdot c_k(t+l) | s^{k,i}(t), w(t)]$ in the context of a DRL agent. This better characterizes the superiority of the DRL agents to maximize future rewards.

D. Quantitative Comparison

Our empirical approach provides explanations by quantitatively comparing the feature weights to the reference feature weights.

Conventional Machine Learning Methods with Forward-Pass

A conventional machine learning method with a forward-pass has three steps: 1) Predict stock returns with machine learning methods using features. 2) Find optimal portfolio weights under predicted stock returns. 3) Build a regression model between portfolio return and features.

$$\begin{aligned}
\hat{y}(t) &= g(f^1(t), \dots, f^K(t)), \\
q^*(t) &= w^*(t) \odot y(t), \\
q^*(t) &= b_0(t) \cdot 1 + b_1(t) \cdot f^1(t) + \dots + b_K(t) \cdot f^K(t) + \epsilon(t), \\
t &= 1, \dots, T,
\end{aligned} \quad (22)$$

where $g(\cdot)$ is the machine learning regression model. $b_k(t)$ is the gradient of the portfolio return to the k -th feature at time slot t , $i = 1, \dots, K$. $y(t)$ is the true price relative vector at time t . $\hat{y}(t)$ is the predicted price relative vector at time t . $w^*(t)$ is the optimal portfolio weight vector defined in (9), where we set the risk aversion parameter to 0.5. Likewise, we define the feature weights $b(t)_k$ by

$$b(t)_k = \sum_{i=1}^N b_k(t) \cdot f^k(t)_i, \quad (23)$$

which is similar to how we define $\beta(t)_k$ and $M^\pi(t)_k$.

Linear Correlations

Both the machine learning methods and DRL agents take profits from their prediction power. We quantify the prediction

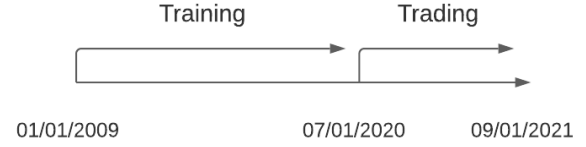


Fig. 3. Data split for the training and trading periods.

power by calculating the linear correlations $\rho(\cdot)$ between the feature weights of a DRL agent and the reference feature weights and similarly for machine learning methods.

Furthermore, the machine learning methods and DRL agents are different when predicting future. The machine learning methods rely on single-step prediction to find portfolio weights. However, the DRL agents find portfolio weights with a long-term goal. Then, we compare two cases, single-step prediction and multi-step prediction.

For each time step, we compare a method's feature weights with $\beta(t)$ to measure the single-step prediction. For multi-step prediction, we compare with a smoothed vector,

$$\beta^W(t) = \frac{\sum_{j=0}^{W-1} \beta(t+j)}{W}, \quad (24)$$

where W is the number of time steps of interest. It is the average reference feature weights over W steps.

For $t = 1, \dots, T$, we use the average values as metrics. For the machine learning methods, we measure the single-step and multi-step prediction power using

$$\begin{aligned}
\bar{\rho}(b, \beta) &= \frac{\sum_{t=1}^T \rho(b(t), \beta(t))}{T}, \\
\bar{\rho}(b, \beta^W) &= \frac{\sum_{t=1}^{T-W+1} \rho(b(t), \beta^W(t))}{T-W+1}.
\end{aligned} \quad (25)$$

For the DRL-agents, we measure the single-step and multi-step prediction power using

$$\begin{aligned}
\bar{\rho}(M, \beta) &= \frac{\sum_{t=1}^T \rho(M(t), \beta(t))}{T}, \\
\bar{\rho}(M, \beta^W) &= \frac{\sum_{t=1}^{T-W+1} \rho(M(t), \beta^W(t))}{T-W+1}.
\end{aligned} \quad (26)$$

In (25) and (26), the first metric represents the average single-step prediction power during the whole trading period. The second metric then measures the average multi-step prediction power.

These two metrics are important to explain the portfolio management task.

- **Portfolio performance:** A closer relationship to the reference model indicates a higher prediction power and therefore a better portfolio performance. Both the single-step prediction and multi-step prediction power are expected to be positively correlated to the portfolio's performance.
- **The advantage of DRL agents:** The DRL agents make decisions with a long-term goal. Therefore the multi-step prediction power of DRL agents is expected to outperform their single-step prediction power.



Fig. 4. The cumulative portfolio return curves of machine learning and DRL models (from 2020-07-01 to 2021-09-01).

- **The advantage of machine learning methods:** The portfolio management strategy with machine learning methods relies on single-step prediction power. Therefore, the single-step prediction power of machine learning methods is expected to outperform their multi-step prediction power.
- **The comparison between DRL agents and machine learning methods:** The DRL agents are expected to outperform the machine learning methods in multi-step prediction power and fall behind in single-step prediction power.

V. EXPERIMENTAL RESULTS

In this section, we describe the data set, compared machine learning methods, trading performance and explanation analysis.

A. Stock Data and Feature Extraction

We describe the stock data and the features.

Stock data. We use the FinRL library [Liu+20] and the stock data of Dow Jones 30 constituent stocks, accessed at the beginning of our testing period, from 01/01/2009 to 09/01/2021. The stock data is divided into two sets. Training data set (from 01/01/2009 to 06/30/2020) is used to train the DRL agents and machine learning models, while trading data set (from 07/01/2020 to 09/01/2021) is used for back-testing the trading performance.

Features. We use four technical indicators as features in our experiments. $\text{leftmargin} = *$

- MACD: Moving Average Convergence Divergence.
- RSI: Relative Strength Index.
- CCI: The Commodity Channel Index.
- ADX: Average Directional Index.

All data and features are measured in a daily time granularity.

B. Compared Machine Learning Methods

We describe the models we use in experiment. We use four classical machine learning regression models [Ped+11]: Support Vector Machine (SVM), Decision Tree Regression (DT), Linear Regression (LR), Random Forest (RF) and two deep reinforcement learning models: A2C and PPO.

C. Performance Comparison

We use several metrics to evaluate the trading performance. $\text{leftmargin} = *$

- **Annual return:** the geometric average portfolio return each year.
- **Annual volatility:** The annual standard deviation of the portfolio return.
- **Maximum drawdown:** The maximum percentage loss during the trading period.
- **Sharpe ratio:** The annualized portfolio return in excess of the risk-free rate per unit of annualized volatility.
- **Calmar ratio:** The average portfolio return per unit of maximum drawdown.
- **Average Correlation Coefficient (single-step):** It measures a model's single-step prediction capability.
- **Average Correlation Coefficient (multi-step):** It measures a model's multi-step prediction capability. We set $W = 20$ in (24).

As shown in by Fig. 4 and Table I, the DRL agent using PPO reached 35% for annual return and 2.11 for Sharpe ratio, which performed the best among all the others. The other DRL agent using A2C reached 34% for annual return and 2.04 for Sharpe ratio. Both of them performed better than the Dow Jones Industrial Average (DJIA), which reached 31.2% for annual return and 2.0 for Sharpe ratio. As for the machine learning methods, the support vector machine method reached the highest Sharpe ratio: 1.53 and the highest annual return: 26.2%. None of the machine learning methods outperformed the Dow Jones Industrial Average (DJIA).

TABLE I
COMPARISON OF TRADING PERFORMANCE.

(2020/07/01-2021/09/01)	PPO	A2C	DT	LR	RF	SVM	DJIA
Annual Return	35.0%	34 %	10.8%	17.6%	6.5%	26.2%	31.2%
Annual Volatility	14.7%	14.9 %	40.1%	42.4%	41.2 %	16.2 %	14.1 %
Sharpe Ratio	2.11	2.04	0.45	0.592	0.36	1.53	2.0
Calmar Ratio	4.23	4.30	0.46	0.76	0.21	2.33	3.5
Max Drawdown	-8.3%	-7.9%	-23.5%	-23.2%	-30.7 %	-11.3 %	-8.9 %
Ave. Corr. Coeff. (single-step)	0.024	0.030	0.068	0.055	0.052	0.034	N/A
Ave. Corr. Coeff. (multi-step)	0.09	0.078	-0.03	-0.03	-0.015	-0.006	N/A

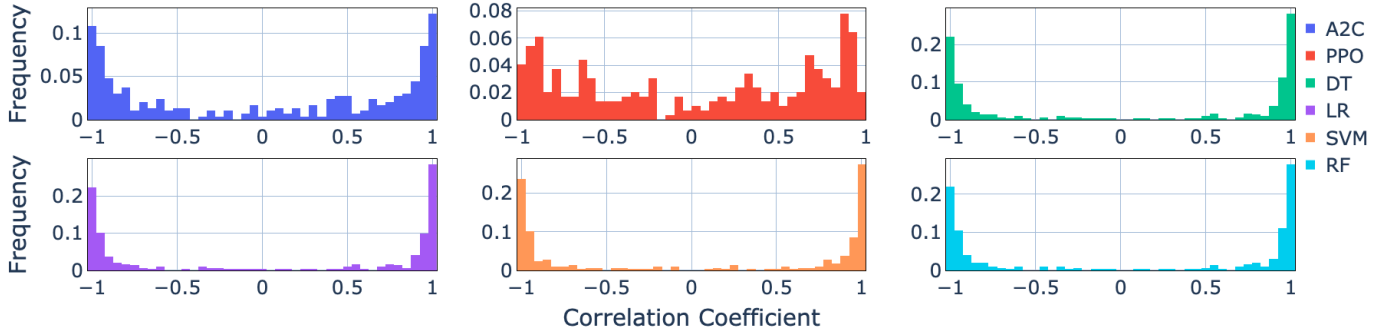


Fig. 5. The histogram of correlation coefficient (single-step) for Advantage Actor Critic (A2C), Proximal Policy Optimization (PPO), Decision Tree (DT), Linear Regression (LR), Support Vector Machine (SVM) and Random Forest (RF).

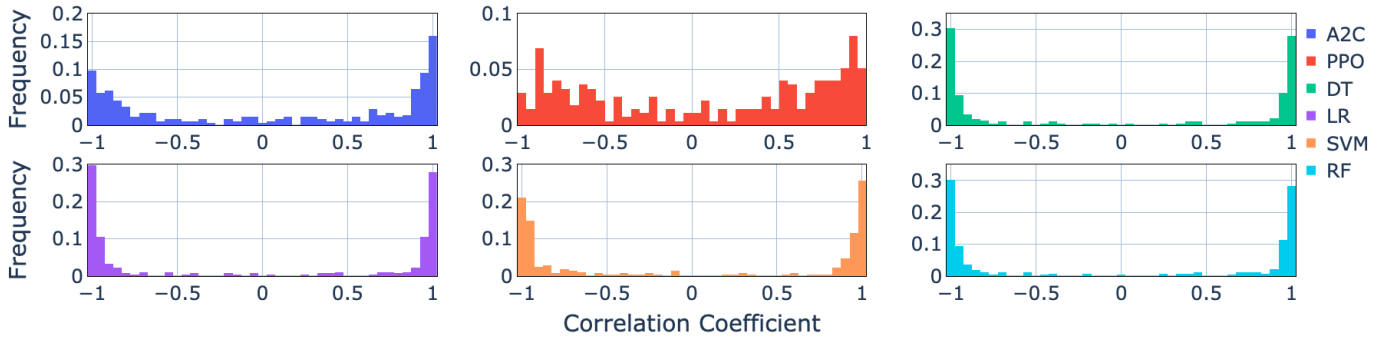


Fig. 6. The histogram of correlation coefficients (multiple-step) for Advantage Actor Critic (A2C), Proximal Policy Optimization (PPO), Decision Tree (DT), Linear Regression (LR), Support Vector Machine (SVM) and Random Forest (RF).

D. Explanation Analysis

We calculate the histogram of correlation coefficients with 1770 samples for 295 trading days. From Fig. 5 and Fig. 6, we visualize the the distribution of correlation coefficients. We derived the statistical tests as in Table II, where “***”, “***” denote significance at the 10% and 5% level. We find that

- The distributions of correlation coefficients are different between the DRL agents and machine learning methods.
- The machine learning methods show greater significance in mean correlation coefficient (single-step) than DRL agents.
- The DRL agents show stonger significance in mean correlation coefficient (multi-step) than machine learning methods.

We show our method empirically explains the superiority

of DRL agents for the portfolio management task. As Fig. 7 shows, the y-axis represents the average coefficients and Sharpe ratio for the whole trading data set, the x-axis represents the model. From Table 1 and Fig. 7, we find that leftmargin=*

- The DRL agent using PPO has the highest Sharpe ratio:2.11 and highest average correlation coefficient (multi-step): 0.09 among all the others.
- The DRL agents’ average correlation coefficients (multi-step) are significantly higher than their average correlation coefficients (single-step).
- The machine learning methods’ average correlation coefficients (single-step) are significantly higher than their average correlation coefficients (multi-step).
- The DRL agents outperform the machine learning meth-

TABLE II
UPPER TAIL TEST TABLE FOR MEAN CORRELATION COEFFICIENT (SINGLE-STEP AND MULTI-STEP) UNDER NULL HYPOTHESIS: THE MEAN CORRELATION COEFFICIENT IS OF NO DIFFERENCE THAN ZERO.

	Z-statistics (single-step)	Z-statistics (multi-step)
PPO	0.6	2.16***
A2C	0.51	1.58**
DT	1.28**	-0.59
LR	1.03	-0.55
RF	0.98	-0.28
SVM	0.64	-0.11

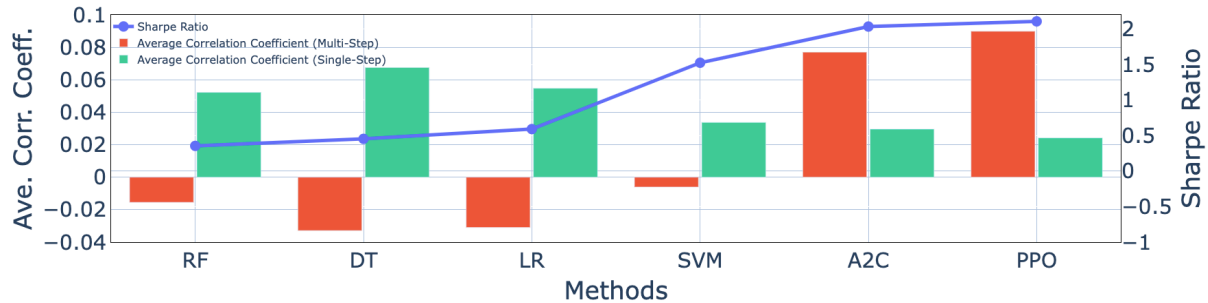


Fig. 7. The comparison of Sharpe ratio and average correlation coefficient.

ods in multi-step prediction power and fall behind in single-step prediction power.

- Overall, a higher mean correlation coefficient (multi-step) indicates a higher Sharpe ratio.

VI. CONCLUSION

In this paper, we empirically explained the DRL agents' strategies for the portfolio management task. We used a *linear model in hindsight* as the *reference model*. We found out the relationship between the reward (namely, the portfolio return) and the input (namely, the features) using integrated gradients. We measured the prediction power using correlation coefficients.

We used Dow Jones 30 constituent stocks from 01/01/2009 to 09/01/2021 and empirically showed that DRL agents outperformed the machine learning models in multi-step prediction. For future work, we will explore the explanation methods for other deep reinforcement learning algorithms and study on other financial applications including trading, hedging and risk management.

REFERENCES

- [FF04] Eugene F Fama and Kenneth R French. "The capital asset pricing model: Theory and evidence". In: *Journal of Economic Perspectives* 18.3 (2004), pp. 25–46.
- [Ped+11] F. Pedregosa et al. "Scikit-learn: Machine learning in Python". In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.
- [ZF14] Matthew D Zeiler and Rob Fergus. "Visualizing and understanding convolutional networks". In: *European conference on computer vision*. Springer, 2014, pp. 818–833.
- [Spr+15] J Springenberg et al. "Striving for simplicity: The all convolutional net". In: *ICLR (workshop track)*. 2015.
- [Mni+16] Volodymyr Mnih et al. "Asynchronous methods for deep reinforcement learning". In: *International Conference on Machine Learning*. PMLR, 2016, pp. 1928–1937.
- [Sel+16] Ramprasaath R Selvaraju et al. "Grad-CAM: Why did you say that?" In: *arXiv preprint arXiv:1611.07450* (2016).
- [Boy+17] Stephen Boyd et al. "Multi-period trading via convex optimization". In: *Foundations and Trends® in Optimization* 3.1 (2017), pp. 1–76.
- [FGX17] Guanhao Feng, Stefano Giglio, and Dacheng Xiu. "Taming the factor zoo". In: *Fama-Miller Working Paper* 24070 (2017).
- [Sch+17] John Schulman et al. "Proximal policy optimization algorithms". In: *arXiv preprint arXiv:1707.06347* (2017).
- [SGK17] Avanti Shrikumar, Peyton Greenside, and Anshul Kundaje. "Learning important features through propagating activation differences". In: *International Conference on Machine Learning*. PMLR, 2017, pp. 3145–3153.
- [Smi+17] Daniel Smilkov et al. "Smoothgrad: removing noise by adding noise". In: *arXiv preprint arXiv:1706.03825* (2017).
- [STY17] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. "Axiomatic attribution for deep networks". In: *International Conference on Machine Learning*. PMLR, 2017, pp. 3319–3328.

- [ACJ19] Akanksha Atrey, Kaleigh Clary, and David Jensen. “Exploratory not explanatory: Counterfactual analysis of saliency maps for deep reinforcement learning”. In: *International Conference on Learning Representations*. 2019.
- [BSN20] Ryan Brown, Harindra de Silva, and Patrick D Neal. “Portfolio performance attribution: A machine learning-based approach”. In: *Machine Learning for Asset Management: New Developments and Financial Applications* (2020), pp. 369–386.
- [Jae+20] Markus Jaeger et al. “Understanding machine learning for diversified portfolio construction by explainable AI”. In: *Available at SSRN 3528616* (2020).
- [Liu+20] Xiao-Yang Liu et al. “FinRL: A deep reinforcement learning library for automated stock trading in quantitative finance”. In: *NeurIPS Workshop on Deep Reinforcement Learning* (2020).
- [Mad+20] Prashan Madumal et al. “Explainable reinforcement learning through a causal lens”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 34. 03. 2020, pp. 2493–2500.
- [TG20] Erico Tjoa and Cuntai Guan. “A survey on explainable artificial intelligence (XAI): Toward medical XAI”. In: *IEEE Transactions on Neural Networks and Learning Systems* (2020).
- [Tom+20] Richard Tomsett et al. “Sanity checks for saliency metrics”. In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 34. 04. 2020, pp. 6021–6029.
- [Con+21] Lin William Cong et al. “AlphaPortfolio: Direct construction through deep reinforcement learning and interpretable AI”. In: *Available at SSRN 3554486* (2021).
- [HCD21] Alexandre Heuillet, Fabien Couthouis, and Natalia Diaz-Rodriguez. “Explainability in deep reinforcement learning”. In: *Knowledge-Based Systems* 214 (2021), p. 106685.
- [Li+21] Zechu Li et al. “FinRL-Podracers: High performance and scalable deep reinforcement learning for quantitative finance”. In: *ACM International Conference on AI in Finance (ICAIF)* (2021).
- [Liu+21a] Xiao-Yang Liu et al. “Data-driven deep reinforcement learning in quantitative finance”. In: *Data-Centric AI Workshop, NeurIPS* (2021).
- [Liu+21b] Xiao-Yang Liu et al. “ElegantRL-podracers: Scalable and elastic library for cloud-native deep reinforcement learning”. In: *Deep RL Workshop, NeurIPS 2021* (2021).
- [Liu+21c] Xiao-Yang Liu et al. “FinRL: Deep reinforcement learning framework to automate trading in quantitative finance”. In: *ACM International Conference on AI in Finance (ICAIF)* (2021).
- [Wik21a] Wikipedia contributors. *Dominated convergence theorem* — *Wikipedia, The Free Encyclopedia*. [Online; accessed 17-September-2021]. 2021. URL: https://en.wikipedia.org/w/index.php?title=Dominated_convergence_theorem&oldid=1037463814.
- [Wik21b] Wikipedia contributors. *Mean value theorem* — *Wikipedia, The Free Encyclopedia*. [Online; accessed 13-September-2021]. 2021. URL: https://en.wikipedia.org/w/index.php?title=Mean_value_theorem&oldid=1036027918.
- [Wik21c] Wikipedia contributors. *Modern portfolio theory* — *Wikipedia, The Free Encyclopedia*. [Online; accessed 13-September-2021]. 2021. URL: https://en.wikipedia.org/w/index.php?title=Modern_portfolio_theory&oldid=1043516653.